

Reviewer Pack — Minimal Symmetry Measure of Boolean Functions over Resolutio...

Entry 5ab4ab52c05b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 08:18:13 UTC

Minimal Symmetry Measure of Boolean Functions over Resolution Proof Trees

Entry ID: 5ab4ab52c05b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 08:18:13 UTC

1. Conjecture

Field A (mathematical branch): Symmetry Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For any boolean function f with n variables, the minimal symmetry measure $\psi(f)$ is upper bounded by $\psi(C) \leq \alpha \cdot \log(\text{size}(C))$, where C is a resolution proof for f and α is an absolute constant.’, ‘ $\psi(f) \geq \log(\text{size}(C))$ for any resolution proof C of a boolean function f with n variables.’, ‘The symmetry measure $\psi(f)$ provides a lower bound on the size of resolution proofs for f .’]

Rationale (proposer’s reasoning):

[‘Symmetry theory has been used to analyze the complexity of certain problems in combinatorics and geometry, but its application to complexity theory is relatively rare. This conjecture proposes that symmetry measures can provide insights into the structure of resolution proof trees, potentially leading to new approaches for proving lower bounds.’, ‘Previous work on resolution proofs often relies on random restrictions or specific constructions, which may not be applicable in general. This conjecture suggests a different approach by using symmetry as an invariant, which is expected to be more robust and general.’, ‘The proposed invariant has the potential to expose hidden structures in resolution proof trees, leading to tighter lower bounds for complexity classes like NP.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4eff5d37df509f34

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given boolean function f with n variables ($n \leq 40$), if the symmetry measure $\psi(f)$ calculated from a resolution proof C for f is less than or equal to α times the logarithm of the size of C , and this condition holds true for at least 24 out of 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symmetry theory AND resolution proof complexity AND Boolean functions
- resolution proof size lower bound AND symmetry measure AND Boolean functions - Boolean
function symmetry measure AND lower bounds on proof size AND resolution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124,
elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def construct_resolution_proof(f):
        n = int(math.log2(len(f)))
        proof = []
        for i in range(n):
            new_clause = []
            for j in range(2**(i+1)):
                if f[j] != f[j ^ (1 << i)]:
                    new_clause.append(j)
                    new_clause.append(j ^ (1 << i))
            proof.append(new_clause)
        return proof

    def calculate_symmetry_measure(proof):
        n = len(proof[0])
        symmetry_count = 0
        for clause in proof:
            for j in range(n):
                if all(clause[i] == clause[(i + j) % n] for i in range(len(clause))):
                    symmetry_count += 1
        return symmetry_count / len(proof)
```

```

def log_size(proof):
    return math.log2(len(proof))

n = random.randint(5, 40)
f = generate_random_boolean_function(n)
C = construct_resolution_proof(f)
psi_C = calculate_symmetry_measure(C)
size_C = len(C)

alpha = 1.0
upper_bound = alpha * log_size(C)

result = {
    "metric_name": "symmetry_measure",
    "metric_value": psi_C,
    "instances_tested": 1,
    "conjecture_holds": psi_C <= upper_bound,
    "counterexample": "" if psi_C <= upper_bound else f"psi(C)={psi_C}, upper_bound={upper_bound}"
}

return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = result["counterexample"]
                first_failing_seed = seed
                break

        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12476
2	propose	ollama_remote	glm4:latest	0	0	12448
3	preregistration	ollama_remote	glm4:latest	0	0	5723
4	novelty	ollama_remote	glm4:latest	0	0	4607
5	novelty	ollama_remote	glm4:latest	0	0	5282
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17013
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8183
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7771
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8071
10	judge	ollama_remote	glm4:latest	0	0	9833

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91408 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5ab4ab52c05b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5ab4ab52c05b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5ab4ab52c05b.tar.gz` (if generated)